

# PROGRAMAÇÃO DE COMPUTADORES

## Funções em Python

Prof. Dr. José Jairo de Santana e Silva

Aulas 7 e 8

# Funções em Python

# AGENDA DA AULA

## ☰ Agenda

### Funções em Python

- O que são funções e por que usá-las
- Definindo funções com **def**
- Parâmetros e argumentos
- Retorno de valores com **return**
- Parâmetros com valor padrão
- Escopo de variáveis (local vs global)

## ☰ Agenda

### Tópicos Avançados e Prática

- Funções com múltiplos retornos
- Funções chamando outras funções
- Funções built-in do Python
- Boas práticas
- Exercícios práticos integrados

## OBJETIVOS DE APRENDIZAGEM

## 🎯 Objetivos

Ao final desta aula, o estudante será capaz de:

- Compreender o conceito de **funções** e sua importância na programação
- Criar funções com **def**, parâmetros e retorno (**return**)
- Utilizar parâmetros com **valores padrão**
- Diferenciar **escopo local** e **escopo global** de variáveis
- Decompor problemas complexos em funções menores e reutilizáveis
- Combinar funções com condicionais e loops para resolver problemas do mundo real

# O QUE SÃO FUNÇÕES?



## ? O que são Funções?

Uma **função** é um bloco de código que realiza uma tarefa específica e pode ser **reutilizado** sempre que necessário.

Analogia do dia a dia:

Pense em uma **receita de bolo**: você define os ingredientes (parâmetros), segue os passos (corpo da função) e obtém o bolo pronto (retorno). Sempre que quiser outro bolo, basta seguir a mesma receita!

Você já usa funções o tempo todo:

```
print("Olá!")      # função built-in  
len("Python")     # função built-in  
int("42")         # função built-in  
input("Digite: ") # função built-in
```

## ❓ Por que usar Funções?

Sem funções (código repetido):

```
# Calcular área do retângulo 1
area1 = 5 * 3
print(f"Área: {area1}")

# Calcular área do retângulo 2
area2 = 8 * 4
print(f"Área: {area2}")

# Calcular área do retângulo 3
area3 = 10 * 6
print(f"Área: {area3}")
```

▮ O mesmo cálculo foi escrito **3 vezes**. E se precisássemos mudar a fórmula?

## ? Por que usar Funções?

Com funções (código reutilizável):

Código:

```
def area_retangulo(base, altura):  
    area = base * altura  
    print(f"Área: {area}")  
  
area_retangulo(5, 3)  
area_retangulo(8, 4)  
area_retangulo(10, 6)
```

Saída:

## Área: 15

## Área: 32

## Área: 60

 Vantagens das Funções

| Vantagem      | Descrição                                              |
|---------------|--------------------------------------------------------|
| Reutilização  | Escreva uma vez, use quantas vezes quiser              |
| Organização   | Código dividido em blocos com responsabilidades claras |
| Manutenção    | Altere em um só lugar, o efeito se propaga             |
| Legibilidade  | Nomes descritivos facilitam a leitura                  |
| Testabilidade | Cada função pode ser testada isoladamente              |

# CRIANDO FUNÇÕES

## </> Sintaxe Básica: **def**

Sintaxe:

```
def nome_da_funcao(parametro1, parametro2):  
    # corpo da função  
    # instruções indentadas  
    return valor # opcional
```

Componentes:

1. **def** - palavra-chave que **define** a função
2. **Nome** - segue as mesmas regras de variáveis (snake\_case)
3. **Parâmetros** - dados que a função recebe (entre parênteses)
4. **Corpo** - bloco indentado com as instruções
5. **return** - valor que a função **devolve** (opcional)

## </> Função Simples (sem parâmetros e sem retorno)

Código:

```
def saudacao():  
    print("=" * 30)  
    print("  Bem-vindo ao sistema!")  
    print("=" * 30)  
  
# Chamando a função  
saudacao()  
print("Continuando o programa...")  
saudacao()
```

Saída:

```
## =====  
##   Bem-vindo ao sistema!  
## =====  
  
## Continuando o programa...  
  
## =====  
##   Bem-vindo ao sistema!  
## =====
```

## </> Função com Parâmetros

**Parâmetros** são variáveis que a função recebe para trabalhar.

Código:

```
def saudacao_personalizada(nome):  
    print(f"Olá, {nome}! Seja bem-vindo(a)!")  
  
saudacao_personalizada("Ana")  
saudacao_personalizada("Carlos")  
saudacao_personalizada("Maria")
```

Saída:

```
## Olá, Ana! Seja bem-vindo(a)!
```

```
## Olá, Carlos! Seja bem-vindo(a)!
```

```
## Olá, Maria! Seja bem-vindo(a)!
```

■ O **parâmetro nome** recebe um valor diferente a cada chamada.



## Função com Múltiplos Parâmetros

Código:

```
def apresentar(nome, idade, curso):  
    print(f"Nome: {nome}")  
    print(f"Idade: {idade} anos")  
    print(f"Curso: {curso}")  
    print("-" * 25)  
  
apresentar("Ana", 20, "Engenharia")  
apresentar("João", 22, "Computação")
```

Saída:

```
## Nome: Ana  
## Idade: 20 anos  
## Curso: Engenharia  
## -----  
  
## Nome: João  
## Idade: 22 anos  
## Curso: Computação  
## -----
```

## 💡 Desafio Didático: Função simples

Crie uma função `linha(tamanho)` que imprime uma linha de `=` com o tamanho informado.

Depois, use essa função para criar um `cabeçalho formatado`:

```
=====
  Relatório de Notas
=====
```

Desafio extra:

- Crie uma função `cabecalho(titulo, tamanho)` que imprime o título centralizado entre as linhas.

# RETORNO DE VALORES

## ↩ O Comando **return**

O **return** faz a função **devolver um valor** para quem a chamou.

Diferença fundamental:

- Função **sem return**: apenas executa ações (imprime, etc.)
- Função **com return**: calcula e **devolve** um resultado

Código:

```
def dobro(numero):  
    return numero * 2  
  
resultado = dobro(7)  
print(f"O dobro de 7 é {resultado}")  
print(f"O dobro de 15 é {dobro(15)}")
```

Saída:

```
## O dobro de 7 é 14
```

```
## O dobro de 15 é 30
```

## ↩ return - Exemplo: Área do Círculo

Código:

```
def area_circulo(raio):  
    pi = 3.14159  
    area = pi * raio ** 2  
    return area  
  
r = 5  
a = area_circulo(r)  
print(f"Raio: {r} -> Área: {a:.2f}")  
print(f"Raio: 10 -> Área: {area_circulo(10):.2f}")
```

Saída:

```
## Raio: 5 -> Área: 78.54
```

```
## Raio: 10 -> Área: 314.16
```

## ↩ **return** - Exemplo: Verificar Par

O **return** pode devolver qualquer tipo, inclusive **booleanos**.

Código:

```
def e_par(numero):  
    return numero % 2 == 0  
  
for i in range(1, 8):  
    if e_par(i):  
        print(f"{i} é par")  
    else:  
        print(f"{i} é ímpar")
```

Saída:

```
## 1 é ímpar  
## 2 é par  
## 3 é ímpar  
## 4 é par  
## 5 é ímpar  
## 6 é par  
## 7 é ímpar
```

## ↩ **return** Encerra a Função

Quando o Python encontra um **return**, a função **para imediatamente**.

Código:

```
def classificar_nota(nota):  
    if nota >= 9:  
        return "A - Excelente"  
    if nota >= 7:  
        return "B - Bom"  
    if nota >= 5:  
        return "C - Regular"  
    return "D - Insuficiente"  
  
print(classificar_nota(9.5))  
print(classificar_nota(7.0))  
print(classificar_nota(3.0))
```

Saída:

## A - Excelente

## B - Bom

## D - Insuficiente

## ↩ Múltiplos Retornos

Uma função pode retornar **vários valores** de uma vez usando vírgula.

Código:

```
def estatisticas(a, b, c):  
    soma = a + b + c  
    media = soma / 3  
    maior = max(a, b, c)  
    menor = min(a, b, c)  
    return soma, media, maior, menor  
  
s, m, mx, mn = estatisticas(8, 5, 10)  
print(f"Soma: {s} | Média: {m:.1f} | Maior: {mx} | Menor: {mn}")
```

Saída:

```
## Soma: 23 | Média: 7.7 | Maior: 10 | Menor: 5
```



## 💡 Desafio Didático: **return**

Crie uma função `calcular_imc(peso, altura)` que:

- Calcule o IMC: `peso / altura ** 2`
- Retorne o valor do IMC e a classificação (string)

Use a função para calcular e exibir o IMC de 3 pessoas.

Desafio extra:

- Crie outra função `relatorio_imc(nome, peso, altura)` que chame `calcular_imc` e imprima um relatório formatado.

## PARÂMETROS COM VALOR PADRÃO

## ≡ Parâmetros com Valor Padrão

Podemos definir **valores padrão** para parâmetros. Se o argumento não for passado, o padrão é usado.

Código:

```
def saudacao(nome, idioma="pt"):
    if idioma == "pt":
        print(f"Olá, {nome}!")
    elif idioma == "en":
        print(f"Hello, {nome}!")
    elif idioma == "es":
        print(f"¡Hola, {nome}!")

saudacao("Ana")           # usa o padrão: "pt"
saudacao("John", "en")    # passa "en"
saudacao("María", "es")   # passa "es"
```

Saída:

```
## Olá, Ana!
```

```
## Hello, John!
```

```
## ¡Hola, María!
```

## ⚙ Valor Padrão - Exemplo: Potência

Código:

```
def potencia(base, expoente=2):  
    return base ** expoente  
  
print(f"3 ao quadrado: {potencia(3)}")      # expoente=2 (padrão)  
print(f"3 ao cubo: {potencia(3, 3)}")      # expoente=3  
print(f"2 elevado a 10: {potencia(2, 10)}") # expoente=10
```

Saída:

```
## 3 ao quadrado: 9
```

```
## 3 ao cubo: 27
```

```
## 2 elevado a 10: 1024
```

Parâmetros com valor padrão devem vir **depois** dos parâmetros obrigatórios.

## Argumentos Nomeados

Podemos passar argumentos pelo **nome**, independente da ordem.

Código:

```
def criar_perfil(nome, idade, cidade="Curitiba"):
    print(f"{nome}, {idade} anos, mora em {cidade}")

# Por posição
criar_perfil("Ana", 20)

# Por nome (qualquer ordem)
criar_perfil(idade=25, nome="Carlos", cidade="São Paulo")

# Misturando (posicionais primeiro)
criar_perfil("Maria", 22, cidade="Londrina")
```

Saída:

```
## Ana, 20 anos, mora em Curitiba
```

```
## Carlos, 25 anos, mora em São Paulo
```

```
## Maria, 22 anos, mora em Londrina
```

# ESCOPO DE VARIÁVEIS

## 👁 Escopo Local vs Global

**Escopo** define onde uma variável é visível e acessível.

- **Variável local**: criada **dentro** da função, só existe ali
- **Variável global**: criada **fora** de qualquer função, acessível em todo o programa

Código:

```
mensagem = "Sou global"      # variável global

def minha_funcao():
    local = "Sou local"       # variável local
    print(mensagem)          # pode acessar a global
    print(local)              # pode acessar a local

minha_funcao()
print(mensagem)              # funciona
# print(local)               # ERRO! 'local' não existe aqui
```

Saída:

```
## Sou global
## Sou local
```

```
## Sou global
```

## 👁 Escopo - Variáveis com Mesmo Nome

Uma variável local **não altera** a global de mesmo nome.

Código:

```
x = 10    # global

def alterar():
    x = 99    # local (nova variável!)
    print(f"Dentro da função: x = {x}")

alterar()
print(f"Fora da função: x = {x}")
```

Saída:

```
## Dentro da função: x = 99
```

```
## Fora da função: x = 10
```

📌 O `x` dentro da função é uma **variável diferente** do `x` global.



## ⚠ A Palavra-chave `global`

Para `modificar` uma variável global dentro de uma função, use `global`.

Código:

```
contador = 0

def incrementar():
    global contador
    contador += 1

incrementar()
incrementar()
incrementar()
print(f"Contador: {contador}")
```

Saída:

```
## Contador: 3
```

▮ **Cuidado:** Evite usar `global` sempre que possível. Prefira passar valores por parâmetros e usar `return`.

# FUNÇÕES NA PRÁTICA

## Funções Chamando Outras Funções

Funções podem **chamar outras funções**, dividindo o problema em partes menores.

Código:

```
def e_primo(n):  
    if n < 2:  
        return False  
    for i in range(2, int(n ** 0.5) + 1):  
        if n % i == 0:  
            return False  
    return True  
  
def listar_primos(limite):  
    primos = []  
    for num in range(2, limite + 1):  
        if e_primo(num):  
            primos.append(num)  
    return primos  
  
resultado = listar_primos(30)  
print(f"Primos até 30: {resultado}")
```

Saída:

```
## Primos até 30: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

## Exemplo: Validação de Dados

Código:

```
def ler_nota(mensagem):  
    while True:  
        nota = float(input(mensagem))  
        if 0 <= nota <= 10:  
            return nota  
        print("Nota inválida! Digite entre 0 e 10.")  
  
def calcular_media(n1, n2, n3):  
    return (n1 + n2 + n3) / 3  
  
def situacao_aluno(media):  
    if media >= 7:  
        return "APROVADO"  
    elif media >= 5:  
        return "RECUPERAÇÃO"  
    return "REPROVADO"
```

Cada função tem **uma responsabilidade**. Juntas, resolvem o problema completo.

 Exemplo: Validação de Dados - Simulação

Saída (simulando):

```
## Nota 1: 8.5
```

```
## Nota 2: 6.0
```

```
## Nota 3: 7.5
```

```
##
```

```
## Média: 7.33
```

```
## Situação: APROVADO
```

## ✖ Funções Built-in Úteis

Python possui diversas funções **prontas para usar**:

| Função               | O que faz                | Exemplo                        |
|----------------------|--------------------------|--------------------------------|
| <code>len()</code>   | Tamanho de uma sequência | <code>len("Python")</code> → 6 |
| <code>max()</code>   | Maior valor              | <code>max(3, 7, 1)</code> → 7  |
| <code>min()</code>   | Menor valor              | <code>min(3, 7, 1)</code> → 1  |
| <code>sum()</code>   | Soma de uma sequência    | <code>sum([1,2,3])</code> → 6  |
| <code>abs()</code>   | Valor absoluto           | <code>abs(-5)</code> → 5       |
| <code>round()</code> | Arredondamento           | <code>round(3.7)</code> → 4    |
| <code>type()</code>  | Tipo do dado             | <code>type(42)</code> → int    |

## ✂ Funções Built-in - Exemplos

Código:

```
numeros = [15, 3, 42, 8, 27, 1]

print(f"Lista: {numeros}")
print(f"Tamanho: {len(numeros)}")
print(f"Maior: {max(numeros)}")
print(f"Menor: {min(numeros)}")
print(f"Soma: {sum(numeros)}")
print(f"Média: {sum(numeros) / len(numeros):.2f}")
```

Saída:

```
## Lista: [15, 3, 42, 8, 27, 1]
```

```
## Tamanho: 6
```

```
## Maior: 42
```

```
## Menor: 1
```

```
## Soma: 96
```

```
## Média: 16.00
```

# BOAS PRÁTICAS



## ✓ Boas Práticas em Funções

### Faça:

- Use nomes descritivos: `calcular_media()` em vez de `cm()`
- Cada função deve ter **uma única responsabilidade**
- Prefira `return` em vez de `print` dentro de funções
- Use parâmetros em vez de variáveis globais
- Mantenha funções **curtas** (idealmente até 20 linhas)

### EVITE:

- Funções que fazem "tudo" (muitas responsabilidades)
- Uso excessivo de `global`
- Funções com muitos parâmetros (mais de 4-5)
- Efeitos colaterais inesperados

## ✓ Exemplo: Bom vs Ruim

## Ruim:

```
def fazer_tudo():  
    n = input("Nome: ")  
    i = int(input("Idade: "))  
    if i >= 18:  
        print(f"{n} é maior")  
    else:  
        print(f"{n} é menor")  
    print("Fim")  
  
fazer_tudo()
```

## Bom:

```
def classificar_idade(idade):  
    if idade >= 18:  
        return "maior de idade"  
    return "menor de idade"  
  
nome = input("Nome: ")  
idade = int(input("Idade: "))  
classe = classificar_idade(idade)  
print(f"{nome} é {classe}")
```

# APLICAÇÕES PRÁTICAS

 Aplicação: Calculadora com Funções

Código:

```
def somar(a, b):  
    return a + b  
  
def subtrair(a, b):  
    return a - b  
  
def multiplicar(a, b):  
    return a * b  
  
def dividir(a, b):  
    if b == 0:  
        return "Erro: divisão por zero!"  
    return a / b  
  
def calcular(a, operador, b):  
    if operador == "+":  
        return somar(a, b)  
    elif operador == "-":  
        return subtrair(a, b)  
    elif operador == "*":  
        return multiplicar(a, b)  
    elif operador == "/":  
        return dividir(a, b)  
    return "Operador inválido"
```

 Calculadora - Saída

Saída (simulando):

```
## 10 + 5 = 15
## 20 - 8 = 12
## 4 * 7 = 28
## 15 / 3 = 5.0
## 10 / 0 = Erro: divisão por zero!
```

 Aplicação: Sistema de Notas - Código

Código:

```
def calcular_media(notas):  
    return sum(notas) / len(notas)  
  
def classificar(media):  
    if media >= 9:  
        return "A"  
    elif media >= 7:  
        return "B"  
    elif media >= 5:  
        return "C"  
    return "D"  
  
def relatorio_aluno(nome, notas):  
    media = calcular_media(notas)  
    conceito = classificar(media)  
    situacao = "Aprovado" if media >= 7 else "Reprovado"  
    print(f"Aluno: {nome}")  
    print(f"Notas: {notas}")  
    print(f"Média: {media:.1f} | Conceito: {conceito} | {situacao}")  
    print("-" * 40)
```

 Sistema de Notas - Saída

Saída:

```
## Aluno: Ana Silva
## Notas: [8.5, 9.0, 7.5]
## Média: 8.3 | Conceito: B | Aprovado
## -----
```

```
## Aluno: Carlos Lima
## Notas: [5.0, 6.5, 4.0]
## Média: 5.2 | Conceito: C | Reprovado
## -----
```

```
## Aluno: Maria Santos
## Notas: [9.5, 10.0, 9.0]
## Média: 9.5 | Conceito: A | Aprovado
## -----
```

## 🔌 Aplicação: Jogo Pedra-Papel-Tesoura

Código:

```
import random

def escolha_computador():
    opcoes = ["pedra", "papel", "tesoura"]
    return random.choice(opcoes)

def verificar_vencedor(jogador, computador):
    if jogador == computador:
        return "Empate!"
    vitorias = {"pedra": "tesoura", "tesoura": "papel", "papel": "pedra"}
    if vitorias[jogador] == computador:
        return "Você venceu!"
    return "Computador venceu!"
```



## 🔧 Pedra-Papel-Tesoura - Saída

Saída (simulando 5 rodadas):

```
## Rodada 1: Você=pedra vs PC=papel -> Computador venceu!  
## Rodada 2: Você=tesoura vs PC=pedra -> Computador venceu!  
## Rodada 3: Você=papel vs PC=papel -> Empate!  
## Rodada 4: Você=pedra vs PC=tesoura -> Você venceu!  
## Rodada 5: Você=tesoura vs PC=pedra -> Computador venceu!
```

 Exercício 1: Funções Básicas

**1a) Saudação** - Crie uma função `saudar(nome, horario)` que exiba "Bom dia", "Boa tarde" ou "Boa noite" conforme o horário (0-11: manhã, 12-17: tarde, 18-23: noite).

**1b) Fatorial** - Crie uma função `fatorial(n)` que calcule e retorne o fatorial de `n` usando um loop `for`.

**1c) Conversão de temperatura** - Crie funções `celsius_para_fahrenheit(c)` e `fahrenheit_para_celsius(f)`. Use-as para converter uma lista de temperaturas.

 Exercício 2: Funções com Retorno

2a) Verificar primo - Crie uma função `e_primo(n)` que retorne `True` ou `False`.

2b) Maior de três - Crie uma função `maior(a, b, c)` que retorne o maior valor **sem usar** `max()`.

2c) Contar vogais - Crie uma função `contar_vogais(texto)` que retorne a quantidade de vogais em uma string.

## Exercício 3: Desafio Integrado

### Sistema de Cadastro de Alunos

Crie um sistema com as seguintes funções:

1. `ler_nota(mensagem)` - lê e valida uma nota (0 a 10)
2. `calcular_media(n1, n2, n3)` - calcula a média de 3 notas
3. `classificar(media)` - retorna "Aprovado", "Recuperação" ou "Reprovado"
4. `exibir_relatorio(nome, notas, media, situacao)` - exibe relatório formatado

O programa principal deve cadastrar vários alunos (usando `while`) e, ao final, exibir a quantidade de aprovados, reprovados e a média geral da turma.

## ☰ Lista de Exercícios para os Alunos

1. Crie uma função que receba um número e retorne se ele é par ou ímpar.
2. Crie uma função que receba dois números e retorne o maior deles.
3. Crie uma função que calcule a área de um triângulo (base e altura).
4. Crie uma função que receba uma string e retorne ela invertida (use `for`).

## ☰ Lista de Exercícios para os Alunos

1. Crie uma função `senha_forte(senha)` que verifique se a senha tem  $\geq 8$  caracteres, pelo menos um número e uma letra maiúscula.
2. Crie uma função `fibonacci(n)` que retorne uma lista com os `n` primeiros termos.
3. Crie funções para uma calculadora completa (+, -, \*, /) com tratamento de divisão por zero.
4. Crie um sistema de caixa de lanchonete usando funções para: exibir cardápio, registrar pedido, calcular total e exibir recibo.

 Resumo da Aula

- **Funções** organizam código em blocos reutilizáveis
- **def** define uma função; **return** devolve um valor
- **Parâmetros** recebem dados; **argumentos** são os valores passados
- **Valores padrão** tornam parâmetros opcionais
- **Escopo local** vs **global**: variáveis locais só existem dentro da função
- Funções podem chamar outras funções (decomposição)
- Prefira funções **curtas**, com **nomes descritivos** e **uma responsabilidade**

## Referências Bibliográficas

- FORBELLONE, A. L. V.; EBERSPACHER, H. F. **Lógica de Programação**: A Construção de Algoritmos e Estruturas de Dados. 4. ed. São Paulo: Pearson, 2016.
- MANZANO, J. A. N. G.; OLIVEIRA, J. F. **Algoritmos**: Lógica para Desenvolvimento de Programação de Computadores. 28. ed. São Paulo: Saraiva, 2020.



## Referências Bibliográficas

- MENEZES, N. N. C. [Introdução a Programação com Python](#): Algoritmos e Lógica de Programação para Iniciantes. 4. ed. Rio de Janeiro: Novatec, 2024.
- Documentação Oficial Python: [docs.python.org/3](https://docs.python.org/3)

# Obrigado!

Prof. Dr. José Jairo de Santana e Silva

Universidade Positivo - 2026/1